



DCGAN FOR STANFORD DOGS DATASET

Aditi
Ashutosh
Dole
(533674)
University of
Messina:
Devices and
circuits for AI

Introduction

DCGAN: Deep Convolutional Generative Adversarial Networks is the one of the most interesting components for artificial intelligence and machine learning. In this, a generator and discriminator are trained simultaneously. The former generates fake images, and the latter distinguishes between fake and real images.

Stanford Dogs Dataset

The Stanford Dogs dataset contains images of 120 breeds of dogs from around the world. This dataset has been built using images and annotation from ImageNet. It contains 20,580 images.



1.Data prep:

1. Load

```
1 # *****
2 # Import
3 # *****
4 import os
5 import cv2
6 import torch
7 import numpy as np
8 import torchvision
9 import matplotlib.pyplot as plt
10 from sklearn import preprocessing
11 from sklearn.model_selection import train_test_split
12
13 # *****
14 # Load data
15 # *****
16
17 if "images" not in os.listdir():
18     os.system("cd ~/github/whisper/images")
19     os.system("wget http://vision.stanford.edu/aditya86/ImageNetDogs/images.tar")
20     os.system("tar -xvf images.tar")
21
22 os.listdir("images")
23
24
```

```
['n02091134-whippet',
'n02093647-Bedlington_terrier',
'n02101006-Gordon_setter',
'n02113712-miniature_poodle',
'n02105096-groenendael',
'n02087394-Rhodesian_ridgeback',
'n02100583-vizsla',
'n02112706-Brabancon_grieffon',
'n02089973-English_foxhound',
'n02112018-Pomeranian',
'n02095570-lakeland_terrier',
'n02109047-Great_Dane',
'n02108422-bull_mastiff',
'n02101556-clumber',
'n02108915-French_bulldog',
'n02111129-Laconberg',
'n02093991-Irish_terrier',
'n02096585-Boston_bull',
'n02105030-collie',
'n02097474-Tibetan_terrier',
'n02088466-bloodhound',
'n02094258-Norwich_terrier',
'n02091244-Ibizan_hound',
'n02107574-Greater_Swiss_Mountain_dog',
'n02106662-German_shepherd',
'n02101388-Brittany_spaniel',
'n02099429-curly-coated_retriever',
'n02095889-Sealyham_terrier',
'n02091035-otterhound']
```

Code

Output

- Data is loaded using the OS library and accessing the source website through terminal commands.
- The folder with images is stored in the directory
- Output displays the breed names of loaded images.

2. Extraction

```
1
2 DATA_DIR = os.getcwd() + "/images/"
3 RESIZE_TO = 100, 100
4 x, y = [], []
5 for i in range(len(os.listdir(DATA_DIR))):
6     for path in [f for f in os.listdir(DATA_DIR+os.listdir(DATA_DIR)[i])]:
7         x.append(cv2.resize(cv2.imread(DATA_DIR + os.listdir(DATA_DIR)[i] + '/' + path), (RESIZE_TO))) #resizing and converting img to numerical array
8         label = os.listdir(DATA_DIR)[i]
9         y.append(label)
10
11 x, y = np.array(x[0:10000]), np.array(y[0:10000])
12 y_label = y
13
14 print("x and y array example: ",x[0:1],y[0:1])
```

- Data_dir gets the current directory with the 'Images' folder
- Resizing parameters are declared
- X , Y empty arrays will contain the images and labels respectively
- The **nested for loop traverses** through the directory , extracts and resizes the images from each labeled folders and assigns them to x and y.
- X, y are then redefined to numerical arrays. First 10,000 images are obtained.

3. Fitting

```
1
2 # Integer encode
3 le = preprocessing.LabelEncoder()
4 le.fit(os.listdir(DATA_DIR))
5 list(le.classes_)
6 y = le.transform(y)
7 print(x.shape, y.shape)

(10000, 100, 100, 3) (10000,)
```

- The code first imports the LabelEncoder class from scikit-learn.
- It then creates a new instance of Label Encoder and passes in the directory where it wants to search for data files.
- The fit method is used to find all the labels in the given directory, which are stored as a list called classes_.
- Next, y is created by calling le.transform on each label and x is created by calling le.classes_ on each label that was found during fit .

4. Splitting

```
1 # *****
2 # Data Split
3 # *****
4
5 x_train, x_test, y_train, y_test = train_test_split(x, y, random_state=1, test_size=0.3)
6 np.save("x_train.npy", x_train); np.save("y_train.npy", y_train);
7 np.save("x_test.npy", x_test); np.save("y_test.npy", y_test);
```

- Training and testing variables are created
- The numpy arrays are saved with respective file names.
- Train_test_split uses args: x,y, random_state(shuffling of data 1 time before splitting), test_size 30%, rest is train_size.

5. Visualising

```
1
2 plt.figure(figsize=(40,10))
3 fig,ax=plt.subplots(2,5)
4 fig.suptitle("Real Images")
5 idx=0
6
7 for i in range(2):
8     for j in range(5):
9         ax[i,j].imshow(x_train[idx])
10        idx+=1
11
12 plt.tight_layout()
13 plt.show()
14
```



Code

output

- Plotting the transformed images

6. Preprocessing input

```
import random
# import tensorflow as tf
from keras.models import Model, Sequential
from keras.layers import Input, Reshape, Dense, Dropout, \
    Activation, LeakyReLU, Conv2D, Conv2DTranspose, \
    Multiply, Embedding, concatenate, \
    MaxPooling2D, UpSampling2D, Flatten, BatchNormalization
from keras.optimizers import Adam, SGD
from keras.preprocessing.image import ImageDataGenerator

X_original = np.load('x_train.npy')

X_real = np.ndarray(shape=(X_original.shape[0], 64, 64, 3))
for i in range(X_original.shape[0]):
    X_real[i] = cv2.resize(X_original[i], (64, 64))

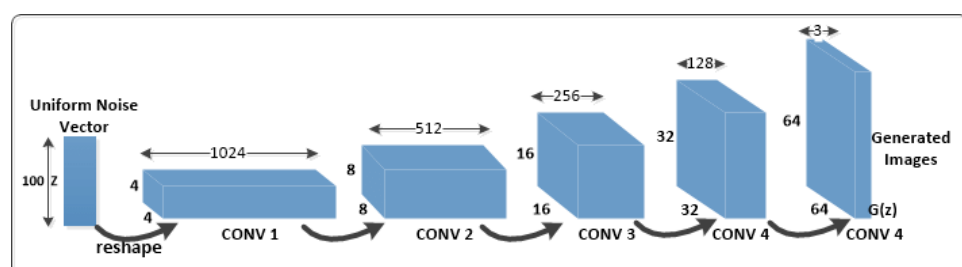
y_real = np.load('y_train.npy')
n_classes = y_real.max() - y_real.min() + 1
img_size = X_real[0].shape

# latent space of noise
z = (100,)
optimizer = Adam(learning_rate=0.0002, beta_1=0.5)
```

- The code starts by loading the data from the file x_train.npy into a NumPy array called X_original.
- Next, it resizes this array to 64x64 and then creates an ndarray of shape (X_original.shape[0], 64, 64, 3) which is used for storing the real image in memory.
- Next, it loads y_train.npy and stores its max() - min() values as n_classes so that we know how many classes there are in our dataset.
- Then it calculates the size of each row in X_real using shape=(X_original.shape[0], 64, 64).
- Lastly, it creates a latent space with 100 dimensions where z will be stored during optimization and optimizer is Adam(learning rate=0.0002, beta1=0).

2. Generator

As mentioned before, the generator is the 'Artist'. It creates fake images from a randomly generated noise vector or matrix and the feedback from Discriminator as input.



The below code is the generator:

- The code starts with a `generator_conv()` function that takes in two inputs, label and noise.
- The first input is an int32 tensor of shape (1,) which represents the number of classes.
- The second input is a z tensor of shape (4 * 4) which represents the probability distribution for each class.
- The code then creates an embedding layer using LeakyReLU to represent each class as a vector in 100 dimensions.
- This layer has four neurons per dimension and one neuron per element in its output space, so it has $4 \times 4 \times 256 = 16384$ neurons total.
- Next, we create another embedding layer using Dense to represent each class as a vector in 100 dimensions with four neurons per dimension and one neuron per element in its output space, so it has $8 \times 8 \times 128 = 65536$ neurons total.
- We use Conv2DTranspose to concatenate these two layers together into one large dense layer with $32 \times 32 \times 128 = 16777216$ neurons total.
- We use BatchNormalization on this new dense layer before passing it through Conv2DTranspose again to get $64 \times 64 \times 128 = 33554432$ neurons total at last!
- Finally we have our model:
- The code is a generator for the following model: `model = Generator(inputs=[noise, label], outputs=generated)`

```
# Build Generator
def generator_conv():
    label = Input((1,), dtype='int32')
    noise = Input(shape=z)

    le = Embedding(n_classes, 100)(label)
    le = Dense(4*4)(le)
    le = Reshape((4, 4, 1))(le)

    noi = Dense(4 * 4 * 256)(noise)
    noi = LeakyReLU(alpha=0.2)(noi)
    noi = Reshape((4, 4, 256))(noi)

    merge = Concatenate()([noi, le])
    ## Size: 4 x 4 x 257

    x = Conv2DTranspose(filters=128,
                        kernel_size=(4, 4),
                        strides=(2, 2),
                        padding='same')(merge)
    x = LeakyReLU(0.2)(x)
    ## Size: 8 x 8 x 128

    # x = BatchNormalization()(x)
    x = Conv2DTranspose(128, (4, 4), strides=(2, 2), padding='same')(x)
    ## Size: 16 x 16 x 128

    x = LeakyReLU(0.2)(x)
    # x = BatchNormalization()(x)

    x = Conv2DTranspose(128, (4, 4), strides=(2, 2), padding='same')(x)
    ## Size: 32 x 32 x 128

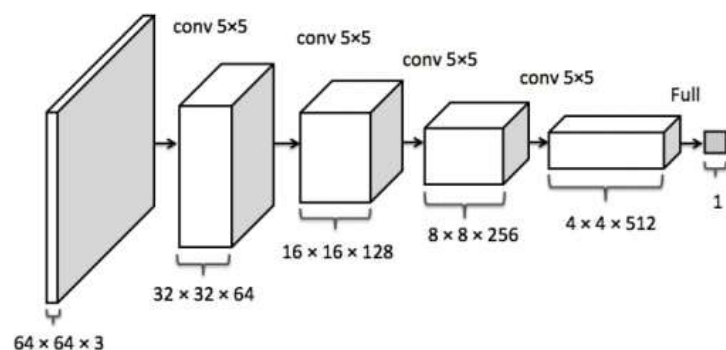
    x = LeakyReLU(0.2)(x)
    # x = BatchNormalization()(x)
    x = Conv2DTranspose(128, (4, 4), strides=(2, 2), padding='same')(x)
    ## Size: 64 x 64 x 128

    x = LeakyReLU(0.2)(x)
    generated = Conv2D(3, (8, 8), padding='same', activation='tanh')(x)
    ## Size: 64 x 64 x 3

    generator = Model(inputs=[noise, label], outputs=generated)
```


3. Discriminator

The discriminator is a classifier of the given image to be real or fake. It uses the output of the generator as training data.



```
# Build Discriminator
def discriminator_conv():
    label = Input((1,)), dtype='int32')
    img = Input(img_size)

    le = Embedding(n_classes, 100)(label)
    le = Dense(img_size[0] * img_size[1])(le)
    le = Reshape((img_size[0], img_size[1], 1))(le)
    ## Size: 64 x 64 x 1
    merge = Concatenate()([img, le])
    ## Size: 64 x 64 x 4
    x = Conv2D(128, kernel_size=(3, 3), strides=(2, 2), padding='same')(merge)
    x = LeakyReLU(0.2)(x)
    ## Size: 32 x 32 x 128
    # x = BatchNormalization()(x)

    x = Conv2D(128, (3, 3), strides=(2, 2), padding='same')(x)
    x = LeakyReLU(0.2)(x)
    ## Size: 16 x 16 x 128
    # x = BatchNormalization()(x)

    x = Conv2D(128, (3, 3), strides=(2, 2), padding='same')(x)
    x = LeakyReLU(0.2)(x)
    ## Size: 8 x 8 x 128
    # x = BatchNormalization()(x)

    x = Conv2D(128, (3, 3), strides=(2, 2), padding='same')(x)
    x = LeakyReLU(0.2)(x)
    ## Size: 4 x 4 x 128
    # x = BatchNormalization()(x)

    x = Conv2D(128, (3, 3), strides=(2, 2), padding='same')(x)
    x = LeakyReLU(0.2)(x)
    ## Size: 2 x 2 x 128
    # x = BatchNormalization()(x)

    x = Flatten()(x)
    x = Dropout(0.4)(x)
    out = Dense(1, activation='sigmoid')(x)

    discriminator = Model(inputs=[img, label], outputs=out)
    discriminator.compile(optimizer=optimizer, loss='binary_crossentropy', metrics=['accuracy'])
    return discriminator
```

- The code starts by defining a function called `discriminator_conv()`.
- This function takes in two inputs, the input image and label.
- The first part of the code is used to create an embedding layer that will be used as a feature map for the model.
- The next step is to reshape this feature map into a 64x64x1 matrix using `Reshape()` followed by concatenating it with another image (the original) using `Concatenate()`.
- After flattening out all layers, we dropout one layer at random and then use `LeakyReLU()` on it before finally adding `BatchNormalization()` on top of it.
- The last step in our process is to build our Discriminator model which consists of `Conv2D` layers stacked together with `LeakyReLU` layers between them.
- We also add `Dropout` layers after each convolutional layer so that they are not overfitting their training data.
- The code is a model that is used to predict the output of an image.
- The `discriminator_conv()` function takes in two input parameters: label and img, which are both integers.
- The `discriminator_conv()` function then creates an embedding layer that maps the input values into a vector of 100 classes.
- This vector is then passed into a dense layer with size 64 x 64 x 1, which will be referred to as le.
- Next, this dense layer will be reshaped into a matrix with size 64 x 64 x 4, which will be referred to as merge.
- Finally, the resulting matrix will have 128 neurons for each dimension and will be called x.

4. Model for Generator and Discriminator to run simultaneously

```
def generator_trainer(generator, discriminator):  
  
    discriminator.trainable = False  
  
    gen_noise, gen_label = generator.input  
    gen_out = generator.output  
    out = discriminator([gen_out, gen_label])  
    model = Model([gen_noise, gen_label], out)  
  
    model.compile(optimizer=optimizer, loss='binary_crossentropy')  
  
    return model
```

- The code starts by declaring a generator and discriminator.
- The generator is the model that will be trained, while the discriminator is used to test whether or not an input was generated from the model.
- Next, it declares two variables: gen_noise and gen_label.
- These are inputs for the generator and outputs for the discriminator respectively.
- Then it declares a variable called out which stores both of these values together as one vector in order to train on them together.
- Finally, it creates a new class called Model with two parameters: noise and label respectively which represent noise and labels in their respective classes (in this case).
- It then compiles this model using optimizer=optimizer, loss='binary_crossentropy'.
- The code will generate a noise and label pair, then use the discriminator to create a model that can predict the correct label.
- The code attempts to train the discriminator on noise-label pairs.

5. Compiling Training function

```
class GAN():  
    def __init__(self, img_shape=(64, 64, 3), latent_space=(100,)):  
        self.img_shape = img_shape # channel_last  
        self.z = latent_space  
        self.optimizer = Adam(0.0002, 0.5)  
  
        self.gen = generator_conv()  
        self.discr = discriminator_conv()  
  
        self.train_gen = generator_trainer(self.gen, self.discr)  
        self.loss_D, self.loss_G = [], []  
  
    def train(self, dataset, epochs=50, batch_size=128): #changed epochs=50  
        # load data  
        imgs, labels = dataset  
        imgs = (imgs - 127.5)/127.5  
        bs_half = batch_size//2  
  
        for epoch in range(int(epochs)): #changed range(epochs)  
            # Get a half batch of random real images  
            idx = np.random.randint(0, imgs.shape[0], bs_half)  
            real_img, real_label = imgs[idx], labels[idx]  
  
            # Generate a half batch of new images  
            noise = np.random.normal(0, 1, size=(bs_half,) + self.z)  
            noise_label = np.random.randint(0, n_classes, bs_half)  
            fake_img = self.gen.predict([noise, noise_label])  
  
            # real = 1 - np.random.random_sample((bs_half, 1)) * 0.1  
            ## One-sided label smoothing  
            real = np.random.uniform(0.9, 1.0, (bs_half, 1))  
            fake = np.zeros((bs_half, 1))  
            ## Random flip 5% labels/data  
            mixpoint = int(bs_half * 0.95)  
            real = np.concatenate([real[:mixpoint], fake[mixpoint:]])  
            fake = np.concatenate([fake[:mixpoint], real[mixpoint:]])  
            np.random.shuffle(real)  
            np.random.shuffle(fake)  
  
            # Train the discriminator  
            loss_fake = self.discr.train_on_batch([fake_img, noise_label], fake)  
            loss_real = self.discr.train_on_batch([real_img, real_label], real)  
            self.loss_D.append(0.5 * np.add(loss_fake, loss_real))  
  
            # Train the generator  
            noise = np.random.normal(0, 1, size=(batch_size,) + self.z)  
            noise_label = np.random.randint(0, n_classes, batch_size)  
            loss_gen = self.train_gen.train_on_batch([noise, noise_label], np.ones(batch_size))  
            self.loss_G.append(loss_gen)  
  
        if ((epoch + 1) * 10) % epochs == 0:  
            print('Epoch (%d / %d): [Loss_D_real: %f, Loss_D_fake: %f, acc: %.2f%%] [Loss_G: %f]' %  
                  (epoch+1, epochs, loss_real[0], loss_fake[0], 100*self.loss_D[-1][1], loss_gen))
```

- The code starts by initializing the **GAN class**.
- The __init__ method takes two parameters: an image shape, and a latent space.
- The first parameter is 64x64x3, which means that it has three channels (red, green, blue).
- This is the size of each channel in pixels.
- The second parameter is 100, which means that this model will be trained on images with 100 dimensions.
- The next line sets the value of z to be a latent space with dimension 100.
- Next comes an optimization algorithm called Adam with hyperparameters 0.0002 and 0.5 for learning rate and momentum respectively; these are used to optimize training loss during training time as well as validation loss during testing time respectively.
- Then there's generator_conv() function that creates a generator from scratch using Adam's optimized parameters; this function returns a generator object whose constructor takes in another generator object as input so we can use it later on when creating our discriminator from scratch using trainer_conv().

- Finally there's `discriminator_conv()`, which creates a discriminator from scratch using Adam's optimized parameters; this function also returns a discriminator object whose constructor takes in another discriminator object as input so we can use it later on when creating our trainable
- The code is a GAN class that takes in an input shape of (64, 64, 3) and returns a latent space of 100.
- The `__init__` method of the GAN class initializes the GAN object with an image size of 64x64x3 channels.
- The z dimension is set to 100 and the optimizer is set to Adam with a learning rate of 0.0002 and momentum 0.5.
- The `generator_conv()` function creates the generator while `discriminator_conv()` creates the discriminator model.
- Finally, both models are trained using `generator_trainer()`.

Def train()

- The code starts by loading the dataset.
- It then creates a list of images and labels for each image in the dataset.
- The code then loops through all of the epochs to get a half batch of random real images, noise, and fake_img.
- The code starts by getting a random index from 0-n_classes-1 where n_classes is equal to bs_half (the number of classes).
- Then it gets the real label for that image and generates a new noise with that label using `np.random()`.
- Next it predicts an image with this new noise using `self.gen()`
- The code will train the model for 50 epochs.
- The images are split into two batches of 128, one half is used to make new images and the other half is used to predict what the label should be.
- The code generates a random noise image with a probability of 0-1 and then predicts it's label from the labels in the dataset.

6. Final Plotting

```
def plt_img(gan):
    r, c = 4, 4
    noise = np.random.normal(0, 1, (r * c, 100))
    noise_label = np.arange(0, c).reshape(-1, 1)
    noise_label = np.dot(noise_label, np.ones((1, r))).astype('int32').T.reshape(-1, 1)
    gen_imgs = gan.gen.predict([noise, noise_label])
    # Rescale images 0 - 1
    gen_imgs = 0.5 * gen_imgs + 0.5
    fig, axs = plt.subplots(r, c)
    cnt = 0
    for i in range(r):
        for j in range(c):
            axs[i,j].imshow(gen_imgs[cnt])
            axs[i,j].set_title("Dog type: %d" % noise_label[cnt])
            axs[i,j].axis('off')
            cnt += 1
    plt.show()
    return
```

- The code starts by defining the variables.
- The first variable is called "gan".
- This is a generator that will be used to generate images of dogs.
- The second variable, r, c, and noise are all integers that define the size of the image in pixels.
- Noise is a random number between 0 and 1 with values ranging from 0 to 100.
- It's used as an input for generating images of dogs using gan.
- The next line defines noise_label which is just a label for each pixel on the image generated by gan so it can be easily identified later on in code when plotting them out on a graph or displaying them in text form.
- Next comes gen_imgs which contains two arrays: one containing noise and one containing noise_label (the labels).
- These arrays are then reshaped into 2D arrays with dimensions equal to r x c so they can be plotted onto axes later on in code without having to do any extra work beforehand like resizing them or converting them into matrices before plotting them out onto axes because this would require more lines of code than what was already written above!
- The code creates a new image with the same dimensions as the original image and then scales it by half.
-

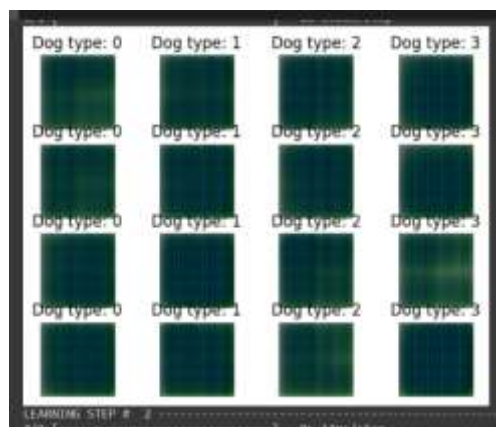
7. Execution

```
1 # train GAN
2 gan = GAN()
3 LEARNING_STEPS = 100
4 BATCH_SIZE = 128
5 # EPOCHS = X_real.shape[0]//BATCH_SIZE
6 EPOCHS = 100

1
2 for learning_step in range(LEARNING_STEPS):
3     print('LEARNING STEP # ', learning_step+1, '-'*50)
4     gan.train([X_real, y_real], epochs=EPOCHS, batch_size=BATCH_SIZE)
5     if (learning_step + 1) % 1 == 0:
6         plt_img(gan)
7
8     if (learning_step + 1) % 30 == 0:
9         gan.gen.save('cgan_generator_%d_v4.h5' % ((learning_step + 1) * 100))
```

- A GAN() object is created.
- There are 100 learning steps and the batch_size is 128. Epochs are defined to 100
- It means the input of 10000 images is divided into 128 sized images.
- Learning Step can be said to be check point for each 100 epochs.
- Epochs are the number of iterations each batch will go through
- After each completed learning step a plot of the generated images will be displayed.

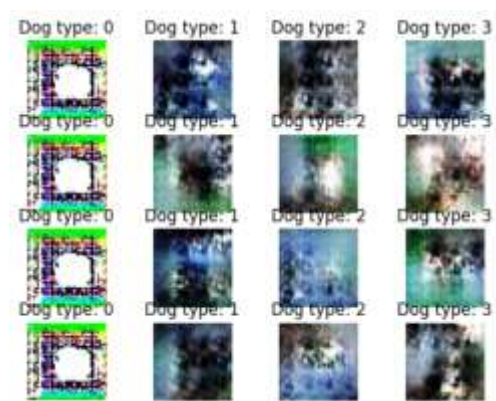
8. Generated images at various Steps :(execution time: 1hr 23 min)



Step 1



Step 10



Step 21



Step 55



Step 87



100

9. Loss in Training:

The accuracy of the discriminator to determine between real and fake images ranges within 5% +- 50%. It means that the discriminator is able to differentiate to a certain extent.

On the other hand, the loss of the generator varies greatly but gradually reduces over learning steps. It means that the generator is failing to produce more noisy images. It stops training after some time.

Step 1

```
Epoch (100 / 100): [loss_D_real: 0.522877, loss_D_fake: 0.070490, acc: 50.00%] [loss_G: 3.808327]
1/1 [-----] - 1s 59ms/step
```

Step 10

```
Epoch (100 / 100): [loss_D_real: 0.633272, loss_D_fake: 0.231461, acc: 46.88%] [loss_G: 5.730557]
1/1 [-----] - 0s 25ms/step
```

Step 21

```
Epoch (100 / 100): [loss_D_real: 0.529475, loss_D_fake: 0.250726, acc: 42.19%] [loss_G: 8.688828]
1/1 [-----] - 0s 20ms/step
```

Step 55

```
Epoch (100 / 100): [loss_D_real: 0.468427, loss_D_fake: 0.209022, acc: 46.88%] [loss_G: 11.661943]
1/1 [-----] - 0s 13ms/step
```

Step 87

```
Epoch (100 / 100): [loss_D_real: 0.416328, loss_D_fake: 0.133062, acc: 46.88%] [loss_G: 37.416767]  
1/1 [=====] - 0s 36ms/step
```

Step 100

```
Epoch (100 / 100): [loss_D_real: 0.512805, loss_D_fake: 0.083086, acc: 49.22%] [loss_G: 24.737568]  
1/1 [=====] - 0s 37ms/step
```

10. References:

1. Base code:

<https://github.com/GH920/Generative-Dog-Images-GAN/blob/master/>

2. Researches:

- a. [Deep Convolutional Generative Adversarial Network | TensorFlow Core](#)
- b. <https://machinelearningmastery.com/how-to-train-stable-generative-adversarial-networks/>
- c. [DCGAN Tutorial — PyTorch Tutorials 2.0.1+cu117 documentation](#)

3. Libraries: Respective Websites

11. Terminologies and Layers:

1. Noise: The noise vector is a random vector that is used as input to the generator network. It is typically drawn from a Gaussian distribution. The purpose of this noise vector is to provide some randomness to the generator so that it can produce different images each time it is run.
2. Embedding layer: This layer is a hidden layer that maps high-dimensional input to low-dimensional. The layer captures the visual features of the images to be used for the model.
3. Dense layer: a dense layer is a layer that is deeply connected with its preceding layer which means the neurons of the layer are connected to every neuron of its preceding layer
4. Leaky ReLU: Rectified Linear Unit is an activation function that will output the input directly if it is positive, otherwise, it will output zero. Leaky ReLU will consider the negative input as well but diminish the impact on the output.
5. Reshape: The Reshape layer can be used to change the dimensions of its input, without changing its data.
6. Convolution: Convolution combines to inputs and gives a single output
7. Conv2DTranspose: the shape of the output of some convolution to something that has the shape of its input while maintaining a connectivity pattern that is compatible with said convolution.
8. Batch Normalization: It is a method used to make training of artificial neural networks faster and more stable through normalization of the layers' inputs by re-centering and re-scaling .
9. Flatten: The Flatten layer is used to convert the input into a one-dimensional array. It is often used in between the convolutional layers and the dense layers in a neural network.
10. Dropout: Dropout randomly drops out (sets to zero) some number of output features of the layer during training. It is a simple way to improve the generalization of deep learning models and reduce overfitting

Libraries and Functions

1. OS: Operating system: Imports the operating system in Colab.	- v System: accesses the command prompt - v Listdir(): lists the directory of the file stored - v Environ[]: sets the PYTHONHASHSEED environment variable to a string value.	
2. cv2 : The code will take an image and convert it to grayscale The code below is a function that takes an image as input, converts it to grayscale, and then returns the output of the conversion.	- v Resize: resizes the images - v Imread: reads the images	
3. torch/ torchvision: These help with the computer vision functions.	- v Utils.make_grid(): creates a grid to place all images according to the length of the respective classes. Most of the classes have a length of 200-300 - v Tensor: creates a multi-dimensional array of the training data.	
4. Numpy: Numerical library for python.	- v Array : one-dimensional matrix - v Save: Saves the file in .npy extension. (numerical converted images_ - v Transpose: transposes the rows and columns - v Unique: Considers distinct data - v Load: load .npy file - v Concatenate: joins the train and test data with the labels - v Random.normal: create random normally distributed noise - v Arange: returns evenly spaced array within the given intervals. - v Ndarray: creates n-dimensional array - v Random.seed: random number generator from given value - v Zeros: matrix of zeros - v Random.ranint: random integers in the range - v Random.uniform: uniformly distributed floating numbers - v Random.shuffle: mixes up the data - v Dot: multiplication of matrices - v Ones: matrix of ones	
5. matplotlib.pyplot: Library to plot the images and information in graphs.	- v Imshow: Shows images - v Show: displays the graph - v Savefig: Saves the generated figure	
6. sklearn : Sci-kit learn module for machine learning	- v Preprocessing - v Model_selection.train_test_split: Allows to split the data into training set and testing set based on the ratio desired	- Ø LabelEncoder - Ø Transform
7. Keras: Neural network layers, models, optimizer, preprocessing module:	- v Models: Model, Sequential: compiles the model, and helps in the sequential execution of the generator, discriminator and the training models - v Layers:	· Input: Entering the input shape of data

		· Reshape: Reshape it to the desired dimensions, · Dense: Gives the output using an activation function for the input and its weight · Dropout: The Dropout layer randomly sets input units to 0 with a frequency of rate at each step during training time, which helps prevent overfitting · Activation: Uses the activation functions like ReLu, sigmoid,etc · LeakyReLU: Rectified Linear unit activation function : returns the maximum for the input and 0 · Conv2D: Creates a kernel using the previous layer and returns a tensor · Conv2DTranspose: Transposes the Conv2D ; changes the direction of the convolution · Multiply: Multiplies a list of inputs · Embedding: Turns positive integers into dense vectors · Concatenate: Concatenates a list of tupkes · MaxPooling2D: Reduces the input, by selecting a max value from a section, and goes through the entire data along its dimension · UpSampling2D: Repeats the rows and columns of the data by size[0] and size[1] · Flatten: Flattens (combines/multiplies) the dimensions of input · BatchNormalization: Transformation to keep the mean output closer to 0 and its standard deviation to 1
	v Optimizers:	· Based on the Adam algorithm is used as it is computationally efficient. · Learning_rate: Schedule function · Beta_1: 1st moment for the exponential decay rate
	v Preprocessing: Allows to simplify the input data by reshaping and standardizing.	

Conclusion

In conclusion, the DCGAN (Deep Convolutional Generative Adversarial Network) applied to the Stanford Dogs dataset has shown impressive results in generating realistic dog images. DCGAN is a powerful generative model that combines convolutional neural networks with adversarial training to produce visually appealing and high-quality images.

By training on the Stanford Dogs dataset, which contains a large number of annotated dog images, the DCGAN model has learned to capture the intricate details, shapes, and textures specific to dogs. The generated images exhibit a remarkable level of realism, which can be improved with the number of epochs. This demonstrates the effectiveness of the DCGAN architecture in capturing the underlying characteristics of the dataset.

DCGAN's ability to generate new, diverse, and visually coherent dog images holds immense potential in various applications. It can be used to augment existing datasets, create synthetic training examples for computer vision tasks related to dogs, or even serve as a creative tool for generating novel dog images for artistic purposes.

However, it is worth noting that the DCGAN model's performance heavily relies on the quality and diversity of the training data. The Stanford Dogs dataset, while comprehensive, may still have its limitations in terms of representing the entire range of dog breeds and variations. As a result, the generated images may exhibit biases or limitations reflective of the dataset's content.

To further enhance the DCGAN's performance on the Stanford Dogs dataset, future research can explore techniques such as progressive training, incorporating additional data sources, or leveraging more advanced generative models. These advancements can lead to even more realistic and diverse dog image generation, pushing the boundaries of what is achievable in generative modeling.

Overall, the DCGAN applied to the Stanford Dogs dataset presents a promising approach for generating high-quality dog images. Its ability to create visually appealing and realistic results has implications in various fields, ranging from computer vision and data augmentation to creative applications.

Thank You!